

VeriGround

Detailed Technical Project Plan

A Real-Time Claim-Level Evidence Verification and Provenance Auditing Framework for RAG Systems

Prepared for: Technical Seminar / Project Review
Department of Computer Science and Business Systems
Panimalar Engineering College

August 2026

1. Purpose of This Document

This document is the single source of truth for the VeriGround build. It fixes the technology stack, the exact pretrained models and open-source components we will use, the algorithm/pipeline that constitutes our actual technical contribution, hardware feasibility on our own machines, deployment options and their cost, and an evaluation plan we can defend in front of a reviewer. Every team member — paperwork group and prototype group — should build against what is written here, so the IEEE paper and the running code never drift apart.

2. Problem Statement and Research Gap

Retrieval-Augmented Generation (RAG) systems reduce hallucination by grounding LLM output in retrieved text, but they do not guarantee that every claim in the final answer is actually supported by that retrieved evidence. Existing verification and evaluation systems each solve part of this problem:

- **ClaimVer, ALCE** — verify claims/citations but are not built around a RAG pipeline's retrieval step.
- **RAGChecker, ARES** — score an entire RAG system's performance, not individual claims a user is reading right now.
- **RAGTruth** — a labeled dataset, not a working system.
- **MetaRAG, RT4CHART** — get close (claim-level, near real-time) but use a 2–3 way label, with no partial-support category, no source-to-document provenance chain, and no live dashboard.

The gap: no existing system combines (a) atomic claim decomposition, (b) evidence alignment against the exact retrieved chunks, (c) a four-way classification that includes Partially Supported, (d) a human-readable explanation per claim, (e) a provenance trail from claim → evidence chunk → source document, and (f) a live auditing dashboard — in one pipeline. VeriGround is built to close exactly that combination.

3. Confirmed Technology Stack

This is the stack we are committing to. Nothing here requires a paid subscription or API key.

Layer	Technology	Why this choice
Frontend	React 18 + Vite + Tailwind CSS + Lucide icons	Already built by Vignesh; fast dev cycle, component-based, matches architecture diagram
Backend / API	Python 3.12 + Flask (REST)	Already built; lightweight, easy to extend route-by-route per module
Embeddings	sentence-transformers — all-MiniLM-L6-v2 (primary)	Free, ~80MB, fast on CPU, industry-standard baseline for semantic similarity
Embeddings (alt/local)	nomic-embed-text via Ollama	Already installed locally; usable as a fallback / for offline demo with zero setup
NLI Classification	microsoft/deberta-v3-base-mnli (primary) or facebook/bart-large-mnli (fallback)	Free, pretrained, HuggingFace transformers pipeline, CPU-workable on our hardware
NER / entity extraction	spaCy — en_core_web_sm	Free, lightweight, already partially wired into nlp_engine.py

Layer	Technology	Why this choice
Explanation generation	Ollama — phi3:mini (3.8B, local)	Free, local, no API cost; generates the human-readable 'why' text per claim
Vector store	FAISS (in-process)	Free, no external DB server needed, runs fine at prototype scale
Results store	SQLite	Free, zero-config, file-based; upgrade path to Postgres if ever needed
Dashboard charts	Recharts (React)	Free, already compatible with existing frontend stack

3.1 Why pretrained models, not training our own

Every paper in our own literature survey — RAGChecker, ARES, MetaRAG, RT4CHART — builds on pretrained embedding and NLI models rather than training new ones from scratch. That is standard, accepted methodology in this exact research area, not a shortcut. Training a competitive NLI or embedding model from scratch would need a large labeled corpus and GPU time we do not have, and would not itself be the point of this project. Our contribution has to sit in what we build on top of these models — which is Section 5.

4. Hardware Feasibility

Checked against the exact machine we're developing and demoing on: ASUS VivoBook, Ryzen 3 3250U, 12GB RAM, no dedicated GPU, 250GB NVMe + 1TB HDD.

Component	Approx. size / RAM	CPU-only latency	Verdict
all-MiniLM-L6-v2 (embeddings)	~80 MB	<50ms per sentence	Comfortable
deberta-v3-base-mnli	~370 MB	~200–400ms per claim/evidence pair	Workable for live demo
bart-large-mnli (fallback)	~1.6 GB	~600ms–1s per pair	Usable but slower — keep as fallback only
spaCy en_core_web_sm	~13 MB	near-instant	Comfortable
Ollama phi3:mini (explanations)	~2.3 GB on disk, ~4GB RAM while loaded	1–3s per explanation	Fine if run once per claim, not in a tight loop
FAISS (flat index, prototype scale)	Negligible at our data sizes	near-instant	Comfortable

Conclusion: everything above runs on our own laptops for a prototype/demo at small batch sizes (a handful of claims per response, which is what a live demo needs). If we ever scaled to production-size traffic, we would need cloud GPU — that is an honest limitation to state, not something to hide.

5. Our Technical Contribution — Weighted Evidence-Fusion Verification Pipeline

This is the answer to "what algorithm/technique did you use to outperform existing systems." We are not claiming a new base model. We are claiming a new fusion pipeline and a four-way decision layer that none of the eight surveyed systems implement together.

5.1 Pipeline steps

- 1. Claim decomposition — split the RAG-generated answer into atomic factual claims (rule-based sentence segmentation + spaCy, refined from the current classify_sentence logic in nlp_engine.py).
- 2. Evidence retrieval — embed each claim and each retrieved evidence chunk with all-MiniLM-L6-v2, get top-k evidence chunks per claim via FAISS cosine similarity.
- 3. NLI scoring — run the claim/evidence pair through deberta-v3-base-mnli, producing entailment, neutral, and contradiction probabilities.
- 4. Entity-overlap scoring — extract named entities, dates, and numbers from the claim and the evidence chunk with spaCy, and compute the overlap ratio between the two sets (our own logic, not from any pretrained model).
- 5. Fusion score — combine the above into a single weighted score (formula below).
- 6. Four-way thresholding — map the fused score to Supported / Partially Supported / Unsupported / Contradicted, instead of NLI's native three labels.
- 7. Explanation generation — pass the claim, evidence, and label to a local Ollama phi3:mini call to produce a one-sentence human-readable justification.
- 8. Provenance write — store claim → evidence chunk id → source document id → timestamp in the results store, surfaced on the live dashboard.

5.2 The fusion formula

Verification Score = $\alpha \cdot \text{SemSim} + \beta \cdot P(\text{entail}) - \gamma \cdot P(\text{contradict}) + \delta \cdot \text{EntityOverlap}$

- SemSim — cosine similarity between claim and evidence embeddings (0–1)
- P(entail), P(contradict) — NLI model's entailment / contradiction probabilities (0–1)
- EntityOverlap — $| \text{entities in claim} \cap \text{entities in evidence} | / | \text{entities in claim} |$ (0–1)
- $\alpha, \beta, \gamma, \delta$ — weights, tuned on our own small labeled validation set (Section 8); starting point $\alpha=0.25, \beta=0.45, \gamma=0.35, \delta=0.15$

5.3 Four-way thresholding (the actual novel piece)

Raw NLI output only gives three buckets, with no room for a claim that is half-right. We define four bands on the fused score:

Fused score range	Label	Meaning
≥ 0.70	Supported	Strong semantic + entailment agreement, high entity overlap
0.40 – 0.69	Partially Supported	Some entailment or overlap, but incomplete match — the case raw 3-way NLI cannot express on its own

Fused score range	Label	Meaning
0.10 – 0.39	Unsupported	Low similarity/entailment, evidence simply doesn't address the claim
< 0.10 or P(contradict) dominant	Contradicted	NLI contradiction signal outweighs entailment/similarity

One line to give the professor: "Existing systems either give a 3-way NLI label with no partial-support case, or score at the whole-system level rather than per claim. We built a weighted fusion pipeline over pretrained embedding, NLI, and entity-overlap signals, and introduced a four-way thresholding scheme — including Partial Support — that no single pretrained model provides out of the box. That's our algorithmic contribution."

6. Module-Wise Technical Breakdown

Module	Input	Technique used	Output	Status
1. AI Content Input	Pasted text / file / URL	Text cleaning, HTML strip, format parsing (file_parser.py)	Cleaned structured text	Built
2. Claim Extraction	Cleaned text	Sentence segmentation (NLTK/regex) + rule-based filter (question/command/greeting/opinion/prediction) — to be layered with spaCy NER for entity signals	List of candidate factual claims	Built (rules) — NER layering pending
3. Evidence Retrieval	Claims + knowledge source	all-MiniLM-L6-v2 embeddings + FAISS top-k search	Top-k evidence chunks per claim	Pending — Week 2
4. Claim-Evidence Verification	Claim + evidence chunks	deberta-v3-base-mnli + entity overlap + fusion formula (Section 5)	Fused score, 4-way label, confidence	Pending — Week 3
5. Explainable AI	Claim, evidence, label	Local Ollama phi3:mini prompt for a one-line justification	Human-readable explanation	Pending — Week 3
6. Provenance / Results Store	Verification results	SQLite write: claim, verdict, score, evidence id, source id, timestamp	Queryable audit trail	Pending — Week 3
7. Live Dashboard	Results store	React + Recharts: grounding score, claim-wise breakdown, source explorer	Real-time auditing UI	Pending — Week 4

7. Local Open-Source LLM Usage (Ollama)

Ollama is used only where a base pretrained NLI/embedding model isn't the right tool — generating natural-language explanations — and deliberately kept out of the classification decision itself, for two reasons: reliability (NLI models are purpose-built and benchmarkable; general LLMs are not) and because a fixed pipeline is easier to defend and evaluate than an LLM's free-text judgment.

Model	Size	Role in VeriGround	Notes
phi3:mini	3.8B, ~2.3GB	Primary — generates the per-claim explanation text	Best latency/quality balance on our hardware

Model	Size	Role in VeriGround	Notes
gemma:2b	2B, ~1.7GB	Fallback if phi3 is too slow during a live demo	Faster, slightly lower explanation quality
nomic-embed-text	~270MB	Optional local alternative to sentence-transformers for embeddings	Useful if we want a fully-offline demo with no HuggingFace download needed live

All three are already pulled locally per our existing Ollama setup — no additional download needed at demo time.

8. Evaluation and Benchmark Plan

To answer "how does this outperform existing approaches" with an actual number, not just an architecture argument:

- Build a small hand-labeled validation set: 30–50 claim/evidence pairs, labeled by us into the four categories, deliberately including borderline "partially supported" cases.
- Run the same set through raw NLI (3-way: entail/neutral/contradict, mapped naively to our 4 labels) as the baseline.
- Run the same set through our fusion pipeline (Section 5).
- Compare accuracy and F1 against our hand labels, and specifically report how many of the "partially supported" cases each approach gets right.
- This comparison — baseline NLI vs our fusion pipeline, on the same 30–50 pairs — is the benchmark result we present.

This is honest about scale: it is a prototype-stage evaluation set, not a published benchmark suite, and we should say so plainly if asked — but it is a real, reproducible comparison, which is what matters at this stage.

9. Deployment Plan

Deployment is not mandatory for the seminar review, but is possible, and entirely free at our current scale.

Component	Free option	Caveat
Frontend (React build)	Vercel free tier or Netlify free tier	Static hosting, generous free limits, no card needed
Backend (Flask + models)	Hugging Face Spaces (Docker) free tier	Better suited than Render for ML-model backends; Render free tier also works but cold-starts and has tighter RAM limits
Vector store / DB	Ships inside the backend (FAISS + SQLite are file-based)	No separate hosted DB needed at this scale

Recommendation: for the live review/demo itself, run everything locally (localhost) rather than depending on a free-tier cold start in front of the mentor — deploy to Hugging Face Spaces / Vercel afterward, as a polish item, once the core pipeline is stable.

10. Cost Summary

Item	Cost
sentence-transformers, spaCy, HuggingFace NLI models	Free (open-source, Apache/MIT-family licenses)
Ollama + local models (phi3:mini, gemma:2b, nomic-embed-text)	Free, runs fully offline
FAISS, SQLite	Free
Vercel / Netlify (frontend hosting)	Free tier sufficient
Hugging Face Spaces (backend hosting)	Free tier sufficient
Total	₹0

11. Risks and Honest Limitations

- CPU-only inference is slow for large batches — fine for a live demo of a handful of claims, not for production traffic.
- No fine-tuning on domain data — our contribution is the fusion pipeline and 4-way thresholding logic, not a new trained model, and we should describe it that way, not oversell it.
- Evaluation set is small (30–50 pairs) and hand-labeled by us — a genuine prototype-stage benchmark, not a peer-reviewed one.
- Fusion weights (α , β , γ , δ) are a reasonable starting point, not yet formally optimized — worth stating if asked, and cheap to tune once the validation set exists.

12. Timeline

Week	Prototype group	Paperwork group
Week 1 (current)	Finish Module 1 & 2; layer spaCy NER into claim filtering; demo-ready	Title, Abstract, Introduction in IEEE format; source 2 base papers + 6–10 reference papers
Week 2	Module 3 — embeddings + FAISS evidence retrieval	Methodology section draft, matched to Section 5/6 of this plan
Week 3	Module 4 & 5 — NLI + fusion scoring + Ollama explanations	System Design + Architecture write-up
Week 4	Module 6 & 7 — provenance store + live dashboard; run evaluation (Section 8)	Results/Evaluation section using benchmark numbers; final formatting pass

13. Quick-Reference Answer for Review Questions

"What algorithm did you use?" — We combine pretrained sentence embeddings (semantic similarity), a pretrained NLI model (entailment/contradiction probability), and our own entity-overlap scoring into a weighted fusion score, then apply a four-way thresholding scheme — including a Partially Supported category — that no

single pretrained model or surveyed system provides natively. That fusion pipeline and thresholding logic is our contribution.

"Why not train your own model?" — Every system in our literature survey (RAGChecker, ARES, MetaRAG, RT4CHART) builds on pretrained embedding/NLI models rather than training from scratch — that is the accepted approach in this exact sub-field. Our novelty is architectural: the fusion of signals and the four-way classification, not a new base model.

"Is this feasible and free?" — Yes — every model and tool used is open-source and free, runs on our own laptop hardware at prototype scale, and deployment (if we choose to) is free-tier on Vercel + Hugging Face Spaces.